

CS781 – AI for Health Sciences

Course Project Update

Anton Rasmussen Spring 2026 Update Date: March 24, 2026

Project Title

Beyond Accuracy Loss: Measuring and Recovering Calibration Degradation in a Quantized Biomedical Large Language Model

1. Title and Brief Introduction

This project investigates whether post-training quantization of BioMistral-7B degrades calibration (measured by Expected Calibration Error, ECE) more than classification accuracy (measured by macro F1), and whether standard post-hoc calibration methods can recover the lost calibration. The work targets two open-access biomedical classification benchmarks—PubMed 20k RCT (5-class sequential sentence classification) and MedNLI (3-class natural language inference)—evaluated across three precision levels (FP16, INT8, INT4) using the bitsandbytes quantization backend.

The core motivation is that existing evaluations of quantized biomedical LLMs focus almost exclusively on aggregate accuracy, which is insufficient for safety-critical clinical deployment. Two additional properties—calibration (whether predicted confidence aligns with actual correctness probability) and prompt robustness (whether predictions are stable across semantically equivalent input phrasings)—are equally important in decision-support contexts. This project goes beyond the known observation that "quantization breaks calibration" to evaluate whether post-hoc methods (temperature scaling, isotonic regression) can *recover* the lost calibration, and at what cost to classification accuracy and prompt stability.

The project hypotheses are:

1. **Primary:** INT4 quantization degrades ECE by a greater relative magnitude than it degrades macro F1 ($|\Delta_{ECE}| > |\Delta_{F1}|$), tested via paired bootstrap comparison.
 2. **Secondary:** Temperature scaling applied after quantization recovers ECE to within 110% of the FP16 baseline without materially reducing accuracy.
 3. **Tertiary:** INT4 quantization increases prompt sensitivity (decreased inter-template agreement via Fleiss' kappa), and this effect is *not* recovered by post-hoc calibration.
-

2. Materials and Method

2.1 Model

BioMistral-7B (Labrak et al., 2024) is an open-weight, 7B-parameter decoder model based on the Mistral architecture, with continued pretraining on biomedical corpora. It is evaluated at three precision levels using the bitsandbytes backend:

Precision	Method	VRAM (approx.)
FP16	Native (torch.float16)	~14 GB
INT8	bitsandbytes LLM.int8() (Dettmers et al., 2022)	~7 GB
INT4	bitsandbytes NF4 (Dettmers et al., 2023)	~3.5 GB

A single quantization backend eliminates cross-backend confounds, ensuring all observed differences reflect precision effects rather than implementation artifacts.

2.2 Datasets

PubMed 20k RCT (Dernoncourt & Lee, 2017): ~200,000 labeled sentences from 20,000 structured abstracts, each classified into one of five rhetorical roles (Background, Objective, Methods, Results, Conclusions). Open access, well-established in biomedical NLP, and its 5-class structure provides richer signal for calibration analysis than binary tasks.

MedNLI (Romanov & Shivade, 2018): ~14,000 premise-hypothesis pairs from clinical text, labeled as entailment, contradiction, or neutral. Distributed via PhysioNet under credentialed access; serves as a complementary 3-class task testing compositional reasoning.

2.3 Probability Extraction

Calibration analysis requires well-defined probability distributions over class labels. The project uses **single-token class code scoring**: each task's labels are mapped to single alphabetic codes (A–E for PubMed RCT, A–C for MedNLI) that are guaranteed to tokenize as exactly one token. For each input, a single forward pass extracts logits at the first generated token position, and softmax is applied over only the code token logits to produce a proper K-dimensional probability distribution.

2.4 Evaluation Metrics

- **Classification:** Macro F1 (primary), per-class F1, accuracy, with 95% bootstrap confidence intervals (n=1,000).
- **Calibration:** ECE with 15 equal-width bins, Adaptive ECE (ACE) with 15 equal-mass bins, reliability diagrams.
- **Calibration recovery:** Temperature scaling (single scalar T, optimized on 15% held-out calibration set) and isotonic regression applied post-hoc.
- **Prompt robustness:** Fleiss' kappa across 5 meaning-preserving prompt templates per task, per-sample flip rate.

2.5 Experimental Design

The full design yields 3 precision levels × 2 tasks × 5 templates = 30 core runs, plus 30 calibration recovery runs (with temperature scaling and isotonic regression). All inference uses greedy decoding

(temperature=0) to eliminate sampling variance. Calibration set isolation is enforced: the held-out calibration split is never used for ECE evaluation.

2.6 Tools and Infrastructure

The project is implemented in Python as the `reliability_eval` package with a YAML-based configuration system supporting hierarchical config composition (base → sweep → component → execution profile). Experiment execution is supported via CLI (`python -m reliability_eval.cli`), standalone scripts (`run_mvp.py`, `run_local.py`, `run_grid.py`), and optional Flyte orchestration for reproducible workflows.

3. Research Results and Progress

3.1 Project Scaffold and Infrastructure (Complete)

The complete project scaffold has been established (PR #1: [Initial scaffold](#)), providing the foundation for all subsequent experimental work:

- **Repository structure:** A full `src/` layout with the `reliability_eval` Python package containing modules for data loading, model management, prompting, inference, metrics, calibration, experiments, reporting, and I/O.
- **Configuration system:** A hierarchical, composable YAML config system (`configs/`) with base, dataset, model, precision, prompt, calibration, sweep, and execution profile configs. The resolution logic deep-merges configs in priority order and supports environment variable overrides.
- **Documentation:** Comprehensive project README, architecture documentation, experiment protocol, execution modes guide, full proposal, and an audit trail (consistency matrix, inventory, and audit comments) for traceability between documentation and implementation.

3.2 Experiment Pipeline (Functional with Mock Inference)

An end-to-end experiment pipeline has been implemented and validated using mock inference:

- **CLI and runners:** Three experiment entrypoints are fully implemented:
 - `run_mvp.py` — Minimal viable product (single PubMed run, mock inference, 8-sample tiny dataset)
 - `run_local.py` — Single experiment from fully resolved YAML config
 - `run_grid.py` — Combinatorial sweep over all config dimensions (with `--dry-run` support)
 - `cli.py` — Unified CLI with `run` and `sweep` subcommands
- **Config resolution:** Configs are resolved by composing base settings, sweep overlays, and component-level configs (dataset, model, precision, calibration, execution profile) into a single resolved config dict per experiment run. This has been tested and validated.
- **Artifact management:** Each experiment run produces a structured output directory containing:
 - `metadata.json` — Run ID, config name, sample count, dataset source, inference mode

- `predictions.jsonl` — Per-example predictions with confidence and full probability distributions
- `metrics.json` — Accuracy, macro F1, per-class F1, ECE
- `resolved_config.yaml` — Full merged config for reproducibility
- `figures/reliability.png` — Reliability diagram

3.3 Metrics Implementation (Partially Complete)

The following metrics are fully implemented and tested:

Metric	Status	Details
Accuracy	✓ Implemented	Simple fraction correct
Macro F1	✓ Implemented	Mean F1 across all classes
Per-class F1	✓ Implemented	Individual class F1 scores
ECE (Expected Calibration Error)	✓ Implemented	15 equal-width bins; weighted sum of
Reliability diagrams	✓ Implemented	Confidence vs. accuracy per bin; PNG output with matplotlib fallback

The following metrics are defined but not yet implemented:

Metric	Status	Notes
ACE (Adaptive Calibration Error)	✗ Not yet implemented	15 equal-mass bins planned
Bootstrap confidence intervals	✗ Not yet implemented	n=1,000 resamples planned
Fleiss' kappa (prompt stability)	✗ Not yet implemented	Inter-template agreement
Per-sample flip rate	✗ Not yet implemented	Template robustness measure

3.4 Prompting and Label Codes (Partially Complete)

- **Label code mappings** are fully implemented for both PubMed RCT (5-class: A–E) and MedNLI (3-class: A–C), ensuring single-token class codes for each task.
- **Prompt rendering** is implemented with template substitution for `{legend}` (label-code mapping) and `{text}` (input sentence).
- **Template registry** loads templates from YAML configs with hardcoded fallback for PubMed templates.
- **Current template count:** 2 templates implemented for PubMed RCT (`pubmed_t1`, `pubmed_t2`); the proposal specifies 5 per task. MedNLI templates are stubbed.

3.5 Mock Inference and MVP Validation

A deterministic mock inference function (`mock_score_example`) has been implemented to validate the full pipeline without requiring GPU resources. The mock scorer:

- Uses seeded randomness (based on `example_id` and prompt text) for reproducibility
- Artificially inflates confidence for the correct label by +0.35 to produce realistic-looking probability distributions
- Returns normalized probabilities over all class codes

This mock inference has been used to validate the complete pipeline end-to-end: config resolution → data loading → prompt rendering → scoring → metrics computation → artifact writing → reliability diagram generation.

3.6 Test Suite

Five test modules have been implemented:

1. `test_config_resolution.py` — Validates YAML config loading and hierarchical merge logic
2. `test_label_code_tokenization.py` — Validates label-to-code mappings for both tasks
3. `test_prompt_rendering.py` — Validates template substitution and legend generation
4. `test_probability_extraction.py` — Validates mock scoring and probability normalization
5. `test_mvp_runner.py` — End-to-end test of the MVP pipeline

3.7 Audit and Consistency Tracking

A consistency audit has been completed, producing three tracking documents:

- **CONSISTENCY_MATRIX.md** — Maps claims in documentation to their implementation status, flagging drift between docs and code (e.g., 5 templates claimed vs. 2 implemented, dataset fallback behavior undocumented).
- **INVENTORY.md** — Catalogs all project artifacts by type (narrative, code, config, data) and status.
- **AUDIT_COMMENTS_AND_PROMPTS.md** — Records specific alignment issues, including prompt template duplication risk (YAML configs vs. hardcoded fallbacks in `render.py`) and stubbed features.

3.8 Summary of Implementation Status

Component	Status	Description
Project scaffold & docs	✔ Complete	Repository structure, README, proposal, architecture docs
Config system	✔ Complete	Hierarchical YAML composition with execution profiles
CLI & experiment runners	✔ Complete	run, sweep, MVP, local, grid entrypoints
Classification metrics	✔ Complete	Accuracy, macro F1, per-class F1

Component	Status	Description
ECE & reliability diagrams	✅ Complete	15-bin ECE, reliability diagram generation
Prompt rendering & label codes	✅ Complete	Template substitution, single-token code mappings
Artifact management	✅ Complete	Structured run directories, metadata, predictions, metrics
Mock inference pipeline	✅ Complete	End-to-end validated with deterministic mock scorer
PubMed data adapter	🟡 Partial	Tiny sample (8 examples) only; full HuggingFace integration pending
Real model loading	❌ Not started	BioMistral-7B load, FP16/INT8/INT4 via bitsandbytes
Real inference (logit extraction)	❌ Not started	Single-token class code scoring from model logits
Post-hoc calibration	❌ Not started	Temperature scaling, isotonic regression
Prompt stability metrics	❌ Not started	Fleiss' kappa, per-sample flip rate
Bootstrap CIs	❌ Not started	n=1,000 resamples on all metrics
MedNLI task	❌ Not started	Dataset loader raises NotImplementedError
Report aggregation	❌ Not started	Multi-run result synthesis

4. Remaining Tasks and Next Steps

4.1 Immediate Priorities (Next 2 Weeks)

Real model loading and inference — This is the critical path item. The project must implement:

1. Loading BioMistral-7B at FP16, INT8, and INT4 precision levels via bitsandbytes.
2. Single-token class code scoring: extracting logits at the first generated token position, selecting code token logits, and computing softmax probabilities.
3. The validation gate from the proposal (Section 5.2): verify on 200 held-out examples that code-token probabilities sum to 1.0, argmax matches greedy output, and reliability diagrams are plausible at FP16.

Full PubMed dataset integration — Replace the 8-example tiny sample with the full PubMed 20k RCT dataset via HuggingFace, including proper train/val/calib/test splits with calibration fraction (15% of validation data).

4.2 Near-Term Tasks (Weeks 3–4)

Post-hoc calibration implementation — Implement temperature scaling (single scalar T, optimized via NLL on calibration set) and isotonic regression (non-parametric, per-class) in the currently stubbed `calibration/` modules.

Prompt template expansion — Expand from 2 to 5 meaning-preserving templates for PubMed RCT, and implement MedNLI templates. Validate all templates on 50 held-out development examples per task.

Bootstrap confidence intervals — Implement n=1,000 bootstrap resampling for all classification and calibration metrics.

Prompt stability metrics — Implement Fleiss' kappa and per-sample flip rate in the `prompt_stability` module.

4.3 Later Tasks (Weeks 5–6)

Full experimental sweep — Run the complete 3 precisions × 2 tasks × 5 templates × pre/post calibration matrix (60 total experiment configurations).

Statistical hypothesis testing — Paired bootstrap comparison of $|\Delta_ECE|$ vs. $|\Delta_F1|$ (primary hypothesis), calibration recovery threshold testing (secondary hypothesis), and prompt stability analysis (tertiary hypothesis).

MedNLI integration — If PhysioNet access is confirmed, implement the MedNLI data loader and extend the pipeline for 3-class inference. If access is delayed, proceed with PubMed RCT as the sole evaluation task (as noted in the proposal fallback plan).

Report and figures — Aggregate multi-run results, generate publication-quality reliability diagrams, calibration recovery plots, and efficiency summary tables. Compile the IEEE-format final report.

4.4 Risk Assessment

Risk	Likelihood	Mitigation
GPU access delays for real inference	Medium	Mock pipeline is fully validated; real inference can begin immediately once hardware is available
MedNLI credentialing delay	Medium	Proposal includes fallback to PubMed-only analysis
INT4 quantization technical issues	Low–Medium	INT8 results provide a complete story; INT4 is additive
Template validation failures	Low	Two templates are already validated; expansion to 5 is incremental

4.5 Revised Timeline

Phase	Original Timeline	Revised Status	Target Completion
Phase 0: Scaffold	Feb 17 – Mar 2	✅ Complete	Done
Phase 1: Infrastructure & data pipeline	Feb 17 – Mar 2	🟡 Partially complete	Mar 31
Phase 2: FP16 baselines & prompt design	Mar 3 – Mar 16	❌ Pending real inference	Apr 7
Phase 3: Quantization experiments	Mar 17 – Apr 6	❌ Pending real inference	Apr 18
Phase 4: Analysis	Apr 7 – Apr 18	❌ Not started	Apr 25
Phase 5: Report & presentation	Apr 19 – May 8	❌ Not started	May 8

The scaffold work was more extensive than initially planned, establishing a robust foundation with full config composition, CLI tooling, and audit trail. While this has shifted the timeline for real inference, the validated mock pipeline ensures that once model loading is implemented, the full experimental sweep can proceed rapidly through the existing infrastructure.

References

- Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *NeurIPS 2022*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized Language Models. *NeurIPS 2023*.
- Dernoncourt, F., & Lee, J.Y. (2017). PubMed 200k RCT: a Dataset for Sequential Sentence Classification in Medical Abstracts. *IJCNLP 2017*.
- Fleiss, J.L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 378–382.
- Guo, C., Pleiss, G., Sun, Y., & Weinberger, K.Q. (2017). On Calibration of Modern Neural Networks. *ICML 2017*.
- Labrak, Y., Bazoge, A., Morin, E., Gourraud, P.-A., Rouvier, M., & Dufour, R. (2024). BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains. *ACL 2024 Findings*.
- Romanov, A., & Shivade, C. (2018). Lessons from Natural Language Inference in the Clinical Domain. *EMNLP 2018*.
- Zadrozny, B., & Elkan, C. (2002). Transforming Classifier Scores into Accurate Multiclass Probability Estimates. *KDD 2002*.